

Lambda Expressions

Functional Programming

Lambda-calculus

- Alonzo Church, ~1935
- The first “programming language”
- The function $f(x, y) := x+y$:

$$\lambda x. \lambda y. (x + y)$$

Lambda-calculus Principles

- A language of **functions**
- In the mathematical sense:
 - same input $\rightarrow$ same output
 - i.e., *stateless*
 - a.k.a. *purity, absence of side-effects, referential transparency*
- No variables, no assignments, no loops!

Functional Languages

- LISP
 - LISt Processor
 - 1958, second programming language after Fortran
 - Scheme
 - Clojure (2007, runs on JVM)
- Haskell (1990)
- ML
 - Meta Language
 - 1973
 - F# (Microsoft .NET, multi-paradigm)
 - OCaml (multi-paradigm)

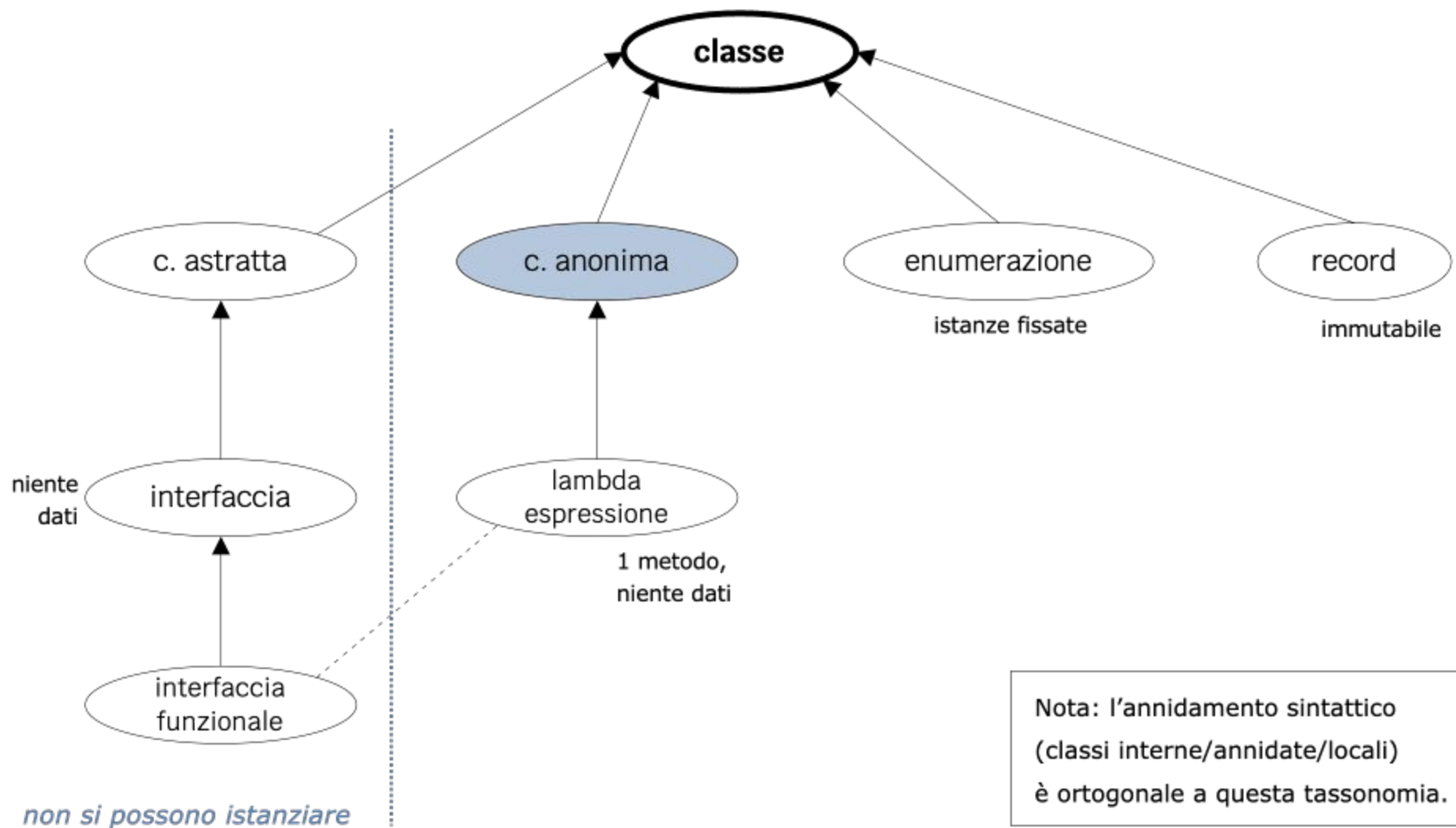
Functional Parallelism

- *Composition* instead of *communication*:

$$f(g(a), h(b), g(c))$$

- Stateless functions g and h can be evaluated *in any order*
- Even in parallel
- Communication only through return values
- No race conditions, no need to synchronize, etc.

Interfaces Get a Boost



Static Methods

- Interfaces have always had `public static final fields`
- Since Java 8, they can also have `public static methods`
- Utility classes like Math could now be interfaces

Default Methods

- Since Java 8, interfaces can contain *default methods*
- A concrete instance method, which can be overridden
- Basically, a regular instance method

Example:

```
interface A {  
    default void foo() {  
        System.out.println("I have an implementation!");  
    }  
}
```

Example: Iterator

- Iterator in Java 7:

```
interface Iterator<T> {  
    boolean hasNext();  
    T next();  
    void remove();  
}
```

- Iterator in Java 8:

```
interface Iterator<T> {  
    boolean hasNext();  
    T next();  
    default void remove() { throw new UnsupportedOperationException(); }  
}
```

Multiple Inheritance

- Java now supports *multiple inheritance of implementation!*

```
interface A {  
    default void foo() {  
        System.out.println("I have an implementation!");  
    }  
}  
  
interface B {  
    default void bar() {  
        System.out.println("I have an implementation too!");  
    }  
}  
  
class X implements A, B { }
```



X inherits both foo and bar

Multiple Inheritance (and its problems)

- Java now supports *multiple inheritance of implementation!*

```
interface A {  
    default void foo() {  
        System.out.println("I have an implementation!");  
    }  
}  
  
interface B {  
    default void foo() {  
        System.out.println("I have another implementation!");  
    }  
}  
  
class X implements A, B { }
```

Ambiguous: compilation error!
X must override foo

Multiple Inheritance (and its problems)

```
interface A {  
    default void foo() {  
        System.out.println("I have an implementation!");  
    }  
}  
  
interface B {  
    default void foo() {  
        System.out.println("I have another implementation!");  
    }  
}  
  
class X implements A, B {  
    @Override  
    public void foo() {  
        A.super.foo();  
    }  
}
```

OK: calls foo from interface A

Default Methods

- Interfaces used to provide *signatures*
- They now also provide *behavior*

So, what's the difference with abstract classes?

Interfaces vs Abstract Classes

- Interfaces are still *stateless* (**no instance fields**)
- Interfaces provide behavior, not state
- Abstract classes provide behavior **and state**

Language Evolution and Backward Compatibility

- You can add static and default methods to an interface *without affecting the implementing classes*
- Many standard interfaces **have been enriched** with new methods
- Collection, List, Comparator, Iterator, etc.

The Comparator Example

- Comparator contains both **static** and **default** methods
- Comparator in Java 7:
 - 1 abstract method (compare)
- Comparator in Java 8:
 - 1 abstract method
 - 9 static methods
 - 7 default methods

The Comparator Example

- A default method in `Comparator<T>`:

```
default Comparator<T> thenComparing(Comparator<? super T> other)
```

- Composes this comparator with another comparator in *lexicographic* order
 - Given two objects to compare...
 - ...first it evaluates `this` comparator...
 - ...if this comparator returns 0, it evaluates the `other` comparator

See also...

- Scalable.java
- Interfaces.java

on <https://bitbucket.org/mfaella/functionaljava>

*Some Interfaces are more
Functional than others*

Functional Interfaces (FIs)

- Any interface with *a single abstract method*
- Static and default methods allowed
- “Functional” because they represent a single function
- Examples:
 - Comparable, Comparator
 - Iterable
 - Runnable
 - *Scalable*

Pure Functional Interfaces

- ...are FIs whose only method has *no side effects*
- ...are FIs meant to be implemented by a *stateless* class
- Examples:
 - Runnable: not pure
 - Comparable: not pure
 - Comparator: **pure**

Pure Functional Interfaces

- Pure FIs respect the *Functional Programming* paradigm
- They play an important role in conjunction with *streams*

The @FunctionalInterface Annotation

- Intended for **pure** FIs
- Compiler checks the “single abstract method” property
- Comparator is annotated with it
- Comparable is *not* annotated with it

Fls in the Java 9 API

- More than 40 pure Fls
- Package `java.util.function`

Examples of Standard FIs

- A function accepting an object

```
public interface Consumer<T> {  
    void accept(T t);  
}
```

- A function producing an object

```
public interface Supplier<T> {  
    T get();  
}
```

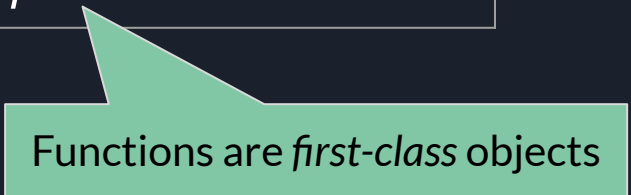
In other languages

- Functional interfaces play the role of *function pointers* in C/C++
- They allow to *pass functions to other functions*
 - Functions as first-class citizens (almost)
 - Typical of functional languages

In other languages

The type of a variable holding a function from int to int:

Language	Type	Description
Java	<code>interface IntFun { int f(int n); }</code>	Functional interface
C/C++	<code>int (*f)(int)</code>	Function pointer
C#	<code>delegate int IntFun(int arg);</code>	Delegate type
ML	<code>int -> int</code>	<i>Just a type</i>



Functions are *first-class* objects

Lambda Expressions

Introducing Lambda Expressions

- A compact syntax to implement FIs
- Lightweight classes representing *anonymous functions*
- Simpler and more efficient than anonymous classes

Example:

```
Comparator<String> byLength =  
    (String a, String b) -> {  
        return Integer.compare(a.length(), b.length());  
    };
```

Lambda Expression Syntax

parameters $\rightarrow$ *body*

Parameters:

<code>(int a, int b)</code>	typed
<code>(a, b)</code>	untyped
<code>(a)</code>	single param
<code>()</code>	no params
<code>a</code>	single param

Body:

<code>{ block }</code>
<code>expr</code>

equiv. to `{ return expr }`

Anonymous functions in other languages

Checking whether a string is longer than 10 characters:

- Java 8: `s -> s.length() > 10`
- ML (functional): `fn s => (size s) > 10`
- C++11: `[] (string s) { return s.length() > 10; }`
- Python: `lambda s: len(s) > 10`

See also...

- Lambda1.java

on <https://bitbucket.org/mfaella/functionaljava>

Typing Lambda Expressions

Type Inference

- Infer a type at compile time
- The compiler filling in a missing type
- Been there since Java 5

Type Inference in Java 5

```
public static <T> T getFirst(T[] array) {  
    return array[0];  
}
```

```
String[] strarray = { "one", "two", "three" };  
String one = getFirst(strarray);
```

- Code does not specify type parameter for “getFirst”
- Still, no cast is needed in assignment
- Type parameter “String” is inferred from the actual parameter “strarray”

Type Inference and Lambdas

The compiler tries to identify:

- the FI being implemented, and
- the parameter types (if omitted)

from the *receiving context* (not from the body of the lambda)

Receiving Contexts

Context must contain enough info to identify the receiving FI

- RHS of assignment

```
Consumer<String> c = lambda
```

- Actual parameter of a method or constructor invocation

```
new Thread(lambda)
```

- Argument of 'return'

```
return lambda
```

- Argument of a cast

```
(Consumer<String>) lambda
```

See also...

- LambdaInference.java

on <https://bitbucket.org/mfaella/functionaljava>

Capturing Values

Capturing Values

- Lambda expressions can access:
 - **static fields** of any class (trivial)
 - **local variables** of enclosing method (needs *capture*)
 - **instance fields** of enclosing object (needs *capture*)

Capturing Locals

- Lambda expressions can access:
 - **local variables** of enclosing method, provided they are *effectively final*
 - (same rule as anonymous classes)

Implementation:

- They store a **copy of that variable**
- This is called “capturing” that variable
- Every runtime evaluation may or may not generate a new object (see example)

Capturing Fields

- Lambda expressions can access:
 - the **enclosing object** (*this*)
 - its **instance fields**

```
class Test {  
    public Consumer<String> foo() {  
        return (msg -> System.out.println(msg + this));  
    }  
}
```

The current **Test** object

Different from anonymous classes!

Capturing Fields

- Lambda expressions can access:
 - **instance fields** of enclosing object

Implementation:

- They store a **reference to the enclosing object**
- They “capture” the current instance: *Instance-capturing lambda expression*
- Similar to capturing the local variable “this”
- Every runtime evaluation generates a new object

Effectively Final Variables

- A variable that is used *as if it was final*
- Not reassigned

Note: it's good practice to declare them *final*

Lambda Expressions vs Anonymous Classes

- Lambdas are more succinct
- Lambdas do not create additional class files
- Not every occurrence of a lambda creates a new object!

On the other hand:

- Anonymous classes can have multiple methods
- Anonymous classes can have state (that is, fields)

See also...

- LambdaImplementation.java
<https://bitbucket.org/mfaella/functionaljava>
- Article “Java 8 Lambdas – A Peek Under the Hood”, by R.Urma and R.Warburton
<https://www.infoq.com/articles/Java-8-Lambdas-A-Peek-Under-the-Hood>